Compressed Representations of Set Collections

Laurea Magistrale in Informatica

Luca Lombardo

29 May 2026

Dipartimento di Informatica
Università di Pisa

Set collections

Definition (Set collection)

A set collection over a finite, totally ordered universe U is a family

$$\mathcal{S} = \{S_1, \ldots, S_m\}, \quad S_i \subseteq U.$$

We write $u = |U|$ and $n = \sum_{i=1}^{m} |S_i|$, and assume $U = [1..u]$.

The encoded object is the whole collection, but each query selects one set S_i and asks for ordered-set information inside it.

- $\mathbf{member}(S_i, x)$: whether $x \in S_i$.
- $\mathbf{access}(S_i, j)$: the j-th element of S_i.
- $\mathbf{rank}(S_i, x)$: the number of elements of S_i at most x.
- $\mathbf{predecessor}(S_i, x)$: the largest element of S_i at most x.
- $\mathbf{successor}(S_i, x)$: the smallest element of S_i at least x.

Counting gives a space target, not a data structure

A representation identifies one object among many possible objects. A bit is redundant when changing it does not change which object is identified.

Proposition (Counting lower bound)

Let $\mathcal{F}$ be a finite class of objects. If every object in $\mathcal{F}$ is represented by a binary codeword of length at most L, then

$$2^L \geq |\mathcal{F}| \quad \text{and therefore} \quad L \geq \lceil \log|\mathcal{F}| \rceil.$$

The bound is only a **space target**. A shortest description may identify the object, but it may give no direct way to test membership, count elements, or select the next one.

We seek representations close to the counting cost that still answer queries.

Independent encoding

Per set. If relations inside $\mathcal{S}$ are ignored, each set is chosen as a subset of the full universe.

$$S_i \subseteq U, \quad |S_i| \text{ fixed} \quad \Longrightarrow \quad \binom{u}{|S_i|} \text{ choices.}$$

Whole collection. Describing the sets independently adds the per-set costs, giving the baseline space for the collection.

$$H_{\mathrm{wc}}(\mathcal{S}) = \sum_{i=1}^{m} \lg \binom{u}{|S_i|}$$

Independent encoding pays for each set in isolation, as if the other sets were not present. Any smaller description must use redundancy across the collection.

Containment exploits nested sets

A first way to use redundancy is to describe a set inside a containing reference.

Definition (Containment step)

$$Q \subseteq P \quad \Longrightarrow \quad \mathrm{cost} = \lg \binom{|P|}{|Q|} \leq \lg \binom{u}{|Q|}.$$

Alanko et al. (2025) build a **containment hierarchy** where each set is represented inside a smallest proper superset, or inside U if no such set exists.

$$U \to \ldots \to P \to Q, \quad Q = P \setminus D.$$

This is query-friendly because every parent-to-child step is a deletion. It becomes compressive when the collection contains a **deep nesting** structure.

Symmetric difference captures close incomparable sets

If neither $A \subseteq B$ nor $B \subseteq A$, containment cannot relate the two sets directly, even when they almost coincide.

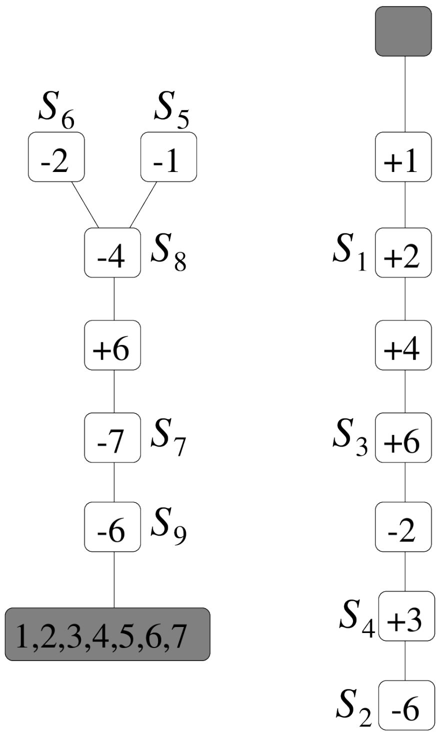
Definition (Symmetric difference)

$$A \triangle B = (A \setminus B) \cup (B \setminus A), \quad |A \triangle B| = |A \setminus B| + |B \setminus A|$$

If S is described from a chosen reference R, only the changed elements are stored as signed edits.

$$+x \quad \text{for} \quad x \in S \setminus R, \quad -y \quad \text{for} \quad y \in R \setminus S$$

Each reference costs $|S \triangle R|$. Gagie, He, and Navarro (2026) choose the references for the whole collection, then store the resulting signed edits on two rooted trees, called **indel trees**.



Choosing references is the main problem

A path representation starts only after each set has received a reference.

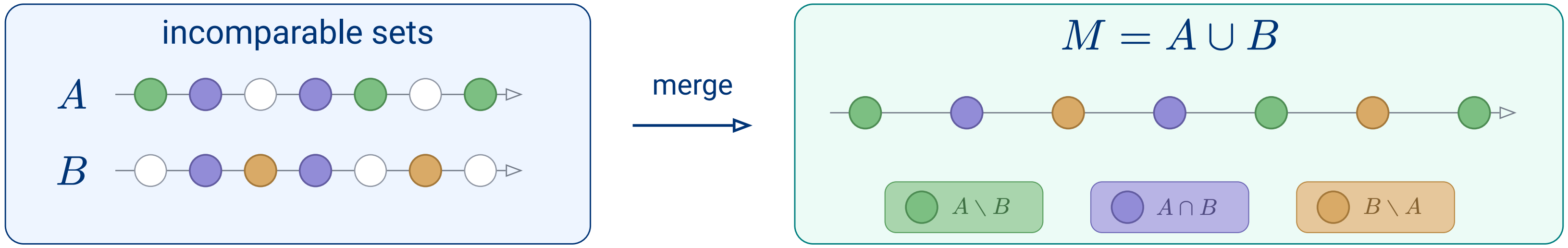
Approach	Reference allowed	Labels paid for
Containment	$P \supseteq S$	deleted elements $P \setminus S$
Symmetric difference	nearby existing reference R	signed edits $S \triangle R$

$$\text{reference domain} \to \text{labels to store} \to \text{queryable paths}$$

Keep deletion paths, but widen the reference domain.

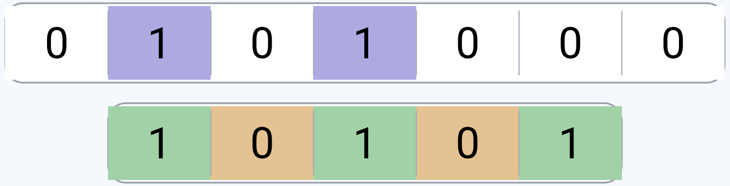
If two input sets overlap but neither contains the other, the containing reference required by deletion paths is missing from the input collection. We create the smallest one: their union.

A merge encodes two sets inside their union



Encode A and B from M

- **First bitvector**: marks $A \cap B$ inside M
- **Second bitvector**: on $M \setminus (A \cap B)$, 1 marks $A \setminus B$

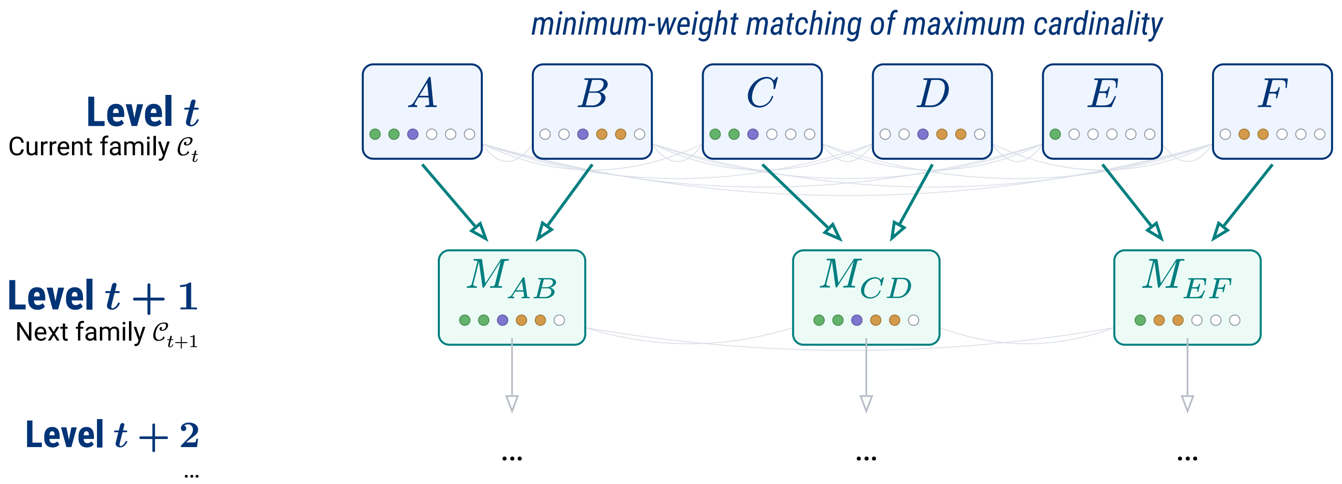


The ordered union M plus two bitvectors determines the pair (A, B). The **cost** is $w(A, B)$

$$w(A,B) = \lg\binom{|M|}{k,l,r} + c(|M|, k) \quad \text{where} \quad k = |A \cap B|, \quad l = |A \setminus B|, \quad r = |B \setminus A|.$$

Choosing unions in the collection

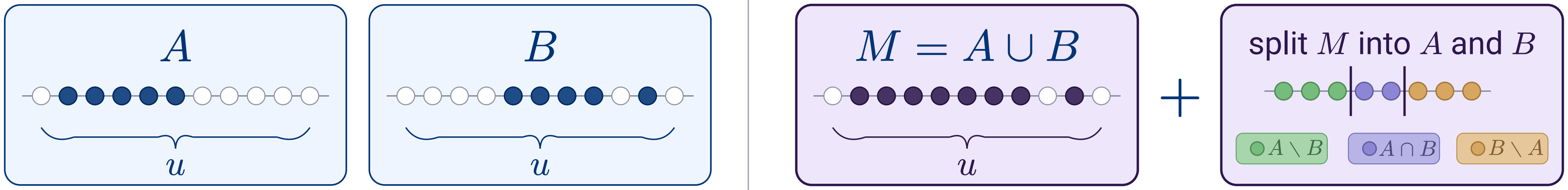
Starting from $\mathcal{C}_0 = \mathcal{S}$, we build a **hierarchy** by choosing pairs $A, B \in \mathcal{C}_t$ and replacing each one by its union $M = A \cup B$ in $\mathcal{C}_{t+1}$.



The construction is determined by the **chosen pairs**. The next step is to make the **cost change** explicit and read when a merge is locally profitable.

Reading the merge score

Old description | New description



store A

$$\lg \binom{u}{|A|}$$

store B

$$\lg \binom{u}{|B|}$$

store the union

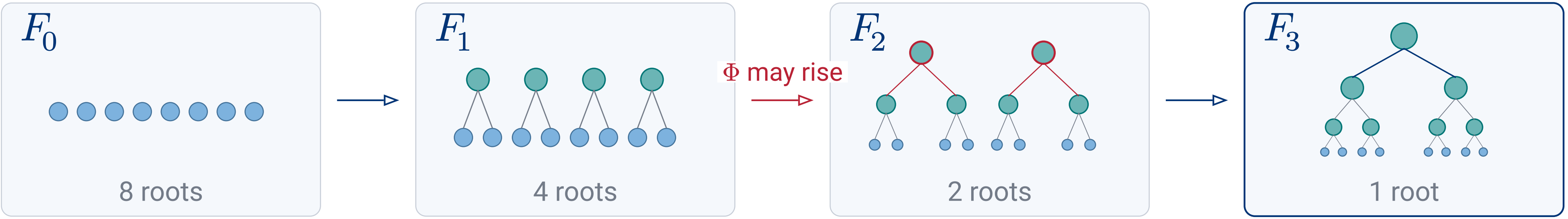
$$\lg \binom{u}{|M|}$$

$$w(A, B)$$

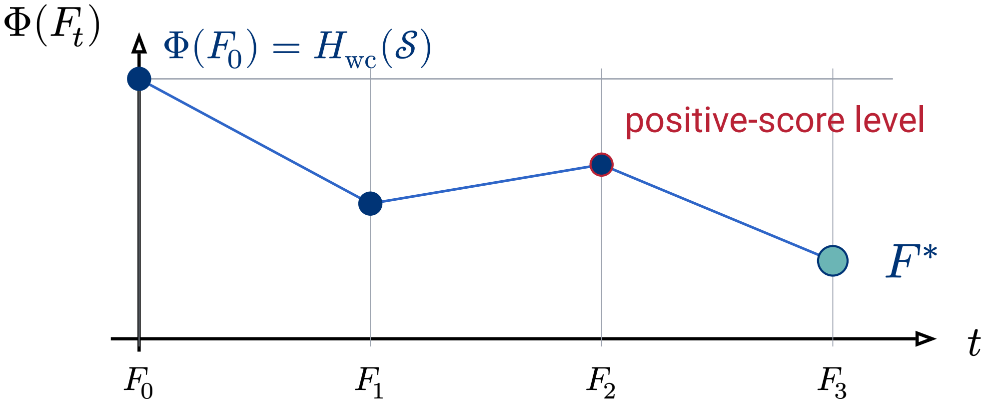
$$\Delta\Phi(A,B) = \underbrace{\lg \binom{u}{|M|}}_{\text{store the union}} + \underbrace{w(A,B)}_{\text{split into } A \text{ and } B} - \underbrace{\lg \binom{u}{|A|}}_{\text{old support of } A} - \underbrace{\lg \binom{u}{|B|}}_{\text{old support of } B} .$$

$\Delta\Phi < 0$: saves bits | $\Delta\Phi = 0$: same counted cost | $\Delta\Phi > 0$: not locally profitable

Greedy matching records the best level



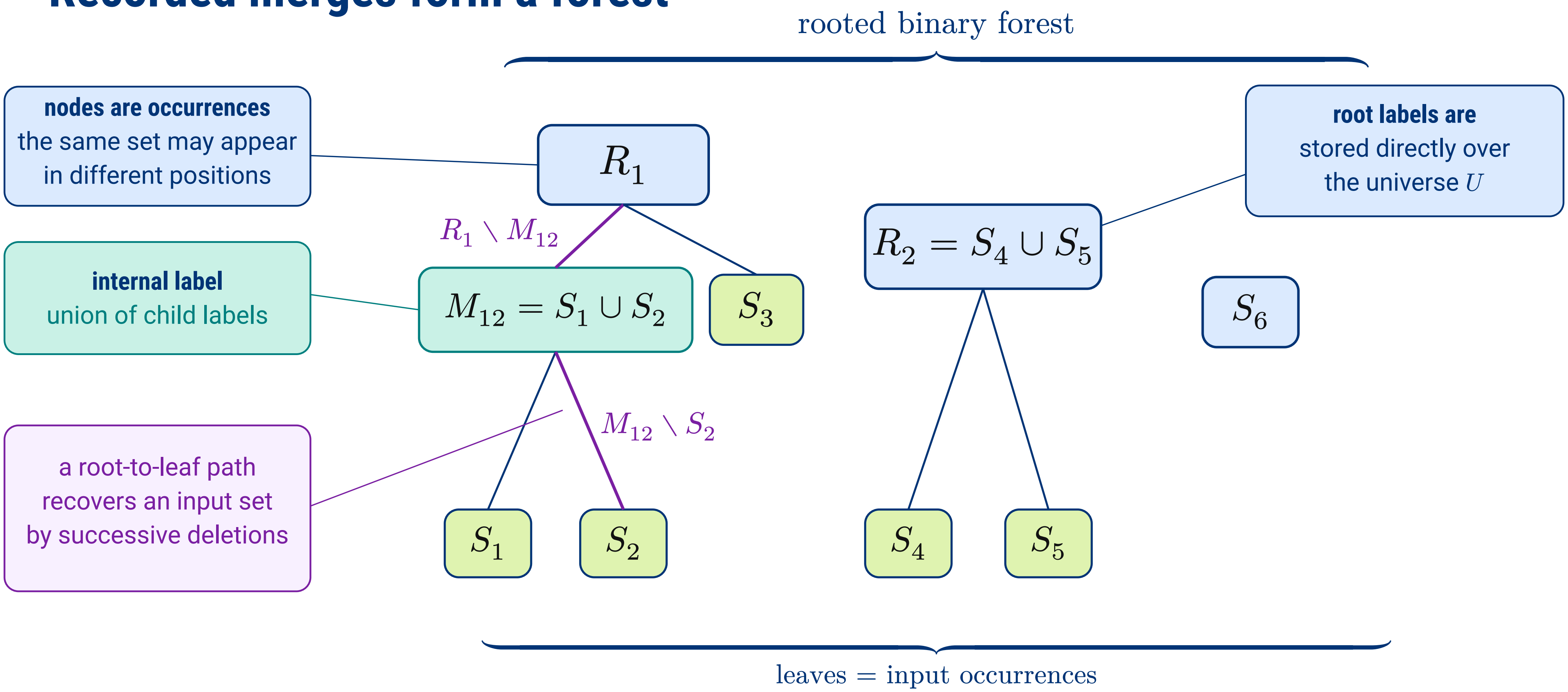
$\Phi(F_0) = H_{\mathrm{wc}}(\mathcal{S})$ | $\Phi(F_1) < \Phi(F_0)$ | $\Phi(F_1) < \Phi(F_2) < \Phi(F_0)$ | $\Phi(F_3) = \min_t \Phi(F_t)$



$$\Phi(F^*) = \min_t \Phi(F_t) \le \Phi(F_0) = H_{\mathrm{wc}}(\mathcal{S}).$$

Because F_0 has no merges, the independent encoding is always among the recorded candidates.

Recorded merges form a forest



Deletion paths

Consider the path from a component root R to the selected input set S, with intermediate sets $P_1, \ldots, P_{t-1}$. Since each parent is a union, this path is a **containment chain**.

$$R = P_0 \supseteq P_1 \supseteq \ldots \supseteq P_t = S$$

For each edge, **store the elements deleted** at that step.

$$D_i = P_{i-1} \setminus P_i$$

The selected input set is recovered from the component root by deleting the elements encountered on the path:

$$S = R \setminus \bigcup_{i=1}^{t} D_i$$

An element deleted on the path cannot reappear below the edge where it is removed.

Queries select survivors on a path

The forest is made queryable by expanding each parent-to-child difference along the path. An edge does not store the child set itself. It stores the elements deleted from the parent.

Every deleted element belongs to the component root R. We label each deletion by its rank in R, so a root-to-leaf path stores the deleted ranks as a subset of $[1..|R|]$.

Thus a query on S is evaluated among the ranks of R that **survive** the path.

$\mathbf{member}(S, x)$	translate x to its rank in R and test whether that rank was deleted.
$\mathbf{rank}(S, x)$	compute $j = \mathrm{rank}_{R(x)}$ and subtract deleted ranks in $[1..j]$.
$\mathbf{access}(S, j)$	select the j-th rank of R that was not deleted on the path.

Membership and rank use the deleted ranks directly, by testing and counting them. Access asks for the complementary set, so it requires **complement selection**: selecting among ranks absent from the deletion path.

What must be stored to query the forest?

Once the SUM forest is fixed, a query on S_i locates the component root R_i and its leaf v_i. It then **switches to ranks** in R_i and reads the deleted ranks on the path to v_i.

Theorem (Representation space)

Root sets have no parent, so each root P_r is stored directly as a rank/select dictionary over U.

$$\sum_{r \ \text{root}} \left(\lg \binom{u}{|P_r|} + o(u) + O(\lg \lg u) \right) \quad \text{bits}$$

The remaining index stores the input directory and the deletion labels expanded along the paths.

$$O\left(m + \sum_{c \ \text{internal}} |P_{\text{left}(c)} \triangle P_{\text{right}(c)}| \right) \quad \text{words}$$

These data support the five selected-set queries.

Query time bounds

After the directory finds (R_i, v_i), the root dictionary translates universe values to ranks in R_i. The remaining cost is a path query over the local deletion alphabet $[1..|R_i|]$.

Theorem (SUM query times)

For nonconstant $|R_i|$, the representation supports:

Operation	*Question on the path*	*Time*
$\mathrm{member}(S_i, x)$	*was the rank of x deleted?*	$O(\lg\lg_\omega |R_i|)$
$\mathrm{rank}(S_i, x)$	*how many deleted ranks lie before x?*	$O(\lg_\omega |R_i|)$
$\mathrm{access}(S_i, j)$	*which rank is the j-th survivor?*	$O\left(\frac{\lg|R_i|}{\lg\lg|R_i|}\right)$
predecessor, successor	*rank followed by access*	*same as access*

These bounds depend on the **component root size** $|R_i|$, not directly on the universe size u. If $|R_i| \ll u$, the logarithmic factors are smaller.

Membership patterns give a lower benchmark

After building a queryable hierarchy, we can ask what any representation is trying to approach. Ignore references for a moment and describe the collection element by element.

Definition (Membership pattern)

$$\sigma(x) = \{i \in [m] : x \in S_i\}.$$

Equal patterns identify elements that no input set can distinguish. For each pattern $\tau \subseteq [m]$, collect all such elements in one region.

$$A_\tau = \{x \in U : \sigma(x) = \tau\}.$$

The regions A_τ partition U. Their sizes count how many times the collection repeats each membership pattern, independently of any chosen hierarchy.

Counting pattern assignments gives a lower bound

Once the pattern regions and their sizes are fixed, the remaining choice is how the labelled elements of U are assigned to those regions.

Definition (Realised patterns and atom sizes)

$$R(\mathcal{S}) = \{\tau \subseteq [m] : A_\tau \neq \emptyset\}, \quad c_\tau = |A_\tau| \ \text{ for } \ \tau \in R(\mathcal{S}).$$

The assignment must respect that exactly c_τ elements receive each realised pattern τ.

Proposition (Atom bound)

With this side information, every lossless representation needs at least

$$L^*(\mathcal{S}) = \lg \binom{u}{(c_\tau)_{\tau \in R(\mathcal{S})}} \ \text{bits.}$$

The atom bound is tight only as a global code

The lower bound is attainable as a global enumerative code.

Proposition (Achievability of L^*)

Given $R(\mathcal{S})$ and $(c_\tau)_{\tau \in R(\mathcal{S})}$, the collection can be encoded in exactly $\lceil L^*(\mathcal{S}) \rceil$ bits.

Order $U = \{x_1, \ldots, x_u\}$. The encoder maps the collection to the word of membership patterns along this order. Its alphabet is $R(\mathcal{S})$, and c_τ fixes how many times each symbol τ appears.

$$\pi(\mathcal{S}) = (\sigma(x_1), \ldots, \sigma(x_u)) \in R(\mathcal{S})^u \quad |\{j : \sigma(x_j) = \tau\}| = c_\tau$$

The code is optimal because it identifies this word among all words with the same counts. It remains global, while SUM is queryable because it exposes local root-to-leaf paths.

Compressed Representations of Set Collections

Thank you for listening!
Any question?

Complement selection

Path selection returns labels present on the path, hence deleted ranks. Access must select a **survivor**, so it asks for a rank absent from that path.

Definition (Complement path selection)

Let $D_{T(v)} \subseteq [1..\sigma]$ be the deletion labels on the root-to-v path. Then $\mathrm{path\_compselect}(v, j)$ returns the j-th smallest label in $[1..\sigma] \setminus D_{T(v)}$.

In an interval J, path counting gives the number $C_{v(J)}$ of deleted labels, so the survivor count is

$$|J| - C_{v(J)}.$$

The same alphabet descent supports complement selection in $O\left(\frac{\lg \sigma}{\lg \lg \sigma}\right)$ time for $\sigma \geq 2$, and in constant time for $\sigma \leq 1$. For SUM, $\sigma = |R_i|$, and access returns

$$\mathrm{select}_{R_i}(\mathrm{path\_compselect}(v_i, j))$$